IBM CERTIFIED DEVELOPER QUANTUM COMPUTATION

Deep-Dive Study Guide • Exam C1000-171

Heavy Code Edition • Qiskit v1.x • Runtime v2 • 2026

DEVELOPER | ALGORITHMS | RUNTIME | ARCHITECTURE

70 questions • 90 minutes • Score 49/70 to pass (70%)

Builds on Associate level: adds Runtime v2, VQE, QAOA, QML, noise, PQC

TABLE OF CONTENTS

SECTION 1

Exam Blueprint & How It Differs from Associate

C1000-171 is the professional-level exam. It assumes C1000-112 knowledge and goes significantly deeper on Runtime primitives, variational algorithms, error mitigation, quantum information theory, and hybrid system design.

Domain	Weight	Depth vs Associate
Advanced Circuit Construction	~20%	Parameterised circuits, circuit library, pulse-level, DD
Qiskit Runtime & Real Hardware	~22%	SamplerV2, EstimatorV2, Sessions, Options, error mitigation
Variational Algorithms & QML	~25%	VQE, QAOA, QNN, quantum kernels, optimisers
Quantum Information Theory	~18%	Entanglement, fidelity, state tomography, Operator class
System Architecture & Design	~15%	Hybrid pipelines, backend selection, cost modelling

Key Difference: The Developer exam requires you to write and reason about COMPLETE algorithms — not just individual API calls. Every practice session should end with a working, end-to-end quantum program.

SECTION 2

Qiskit Runtime v2 – Sampler, Estimator, Sessions

Qiskit Runtime v2 introduces two primitive abstractions that replace the old execute() API. SamplerV2 is for measurement-based tasks (counts, probabilities). EstimatorV2 is for expectation values — the core of VQE and variational methods.

SamplerV2 — Full Usage

```
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler
from qiskit_ibm_runtime import Options
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

# ■■ On simulator ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sim = AerSimulator()
qc = QuantumCircuit(3, 3)
qc.h(0); qc.cx(0,1); qc.cx(1,2); qc.measure_all()
tqc = transpile(qc, sim)

sampler = Sampler(sim)
job = sampler.run([tqc], shots=4096)
result = job.result()

# SamplerV2 result structure (different from v1!)
pub_result = result[0] # PubResult for first PUB
data = pub_result.data # DataBin
counts = data.meas.get_counts() # dict {'000':~2048,'111':~2048}
bitarray = data.meas # BitArray object
print(bitarray.get_counts())
print(bitarray.get_bitstrings()) # list of individual shot results

# ■■ Multiple circuits in one job (batching) ■■■■■■■■■■■■■■■■
qc_a = QuantumCircuit(2,2); qc_a.h(0); qc_a.cx(0,1); qc_a.measure_all()
qc_b = QuantumCircuit(2,2); qc_b.x(0); qc_b.measure_all()
tqc_a = transpile(qc_a, sim)
tqc_b = transpile(qc_b, sim)

job = sampler.run([tqc_a, tqc_b], shots=2048)
res = job.result()
counts_a = res[0].data.meas.get_counts()
counts_b = res[1].data.meas.get_counts()

# ■■ On real hardware with Options ■■■■■■■■■■■■■■■■■■■■■■■■■
service = QiskitRuntimeService(channel='ibm_quantum')
backend = service.least_busy(min_num_qubits=5)

from qiskit_ibm_runtime import SamplerOptions
options = SamplerOptions()
options.dynamical_decoupling.enable = True # enable DD
```

```
options.twirling.enable_gates = True # enable Pauli twirling
options.twirling.num_randomizations = 300

sampler = Sampler(backend, options=options)
tqc = transpile(qc, backend, optimization_level=3)
job = sampler.run([tqc], shots=8192)
print(job.job_id()) # save for retrieval
```

EstimatorV2 — Expectation Values for VQE

```
from qiskit_ibm_runtime import EstimatorV2 as Estimator
from qiskit.quantum_info import SparsePauliOp
from qiskit import QuantumCircuit, transpile
from qiskit.circuit import Parameter
from qiskit_aer import AerSimulator
import numpy as np

sim = AerSimulator()

# Define a Hamiltonian as SparsePauliOp
# H = 0.5*ZZ - 0.5*XX (simple 2-qubit Hamiltonian)
H = SparsePauliOp.from_list([('ZZ', 0.5), ('XX', -0.5)])

# Define parameterised ansatz circuit
theta = Parameter('theta')
qc = QuantumCircuit(2)
qc.ry(theta, 0)
qc.cx(0, 1)
qc.ry(theta, 1)

estimator = Estimator(sim)

# Single evaluation
tqc = transpile(qc.assign_parameters({theta: 0.5}), sim)
job = estimator.run([(tqc, H)])
result = job.result()
ev = result[0].data.evs # expectation value (scalar)
print(f'E(0.5) = {ev:.4f}')

# Sweep over parameter values
theta_vals = np.linspace(0, 2*np.pi, 50)
pubs = []
for t in theta_vals:
bound = qc.assign_parameters({theta: t})
tbound = transpile(bound, sim)
pubs.append((tbound, H))

job = estimator.run(pubs)
energies = [job.result()[i].data.evs for i in range(len(pubs))]
min_energy = min(energies)
print(f'Minimum energy: {min_energy:.4f}')
```

Sessions — Reserving Backend Access

```
from qiskit_ibm_runtime import Session, SamplerV2 as Sampler
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService(channel='ibm_quantum')
backend = service.least_busy(min_num_qubits=5)

# Session keeps connection to backend — reduces queue overhead
# for iterative algorithms (VQE, QAOA parameter sweeps)
with Session(backend=backend) as session:
sampler = Sampler(session=session)
estimator = Estimator(session=session)

# All jobs in this block share the session
# (same backend, same QPU reservation)
job1 = sampler.run([tqc1], shots=4096)
job2 = estimator.run([(tqc2, hamiltonian)])
res1 = job1.result()
res2 = job2.result()
# Session automatically closed on exit

# Session mode options:
# - 'dedicated' : exclusive QPU access (premium)
# - 'batch' : submit multiple jobs, system batches them
# Use Session for iterative variational algorithms — the classical
# optimiser loop requires many sequential quantum evaluations
```

Architect Note: Use Sessions for ALL variational algorithms (VQE, QAOA). Without a session, each iteration queues independently — adding minutes of overhead per iteration on busy backends.

SECTION 3

Advanced Circuit Techniques

Dynamical Decoupling (DD)

DD mitigates decoherence during idle periods by applying symmetric pulse sequences that average out low-frequency noise. The most common sequence is XX (two X gates evenly spaced in the idle window).

```
from qiskit.transpiler.passes import PadDynamicalDecoupling
from qiskit.circuit.library import XGate
from qiskit.transpiler import PassManager, InstructionDurations
from qiskit import transpile

# Enable via Runtime Options (recommended for real hardware)
from qiskit_ibm_runtime import SamplerOptions
opts = SamplerOptions()
opts.dynamical_decoupling.enable = True
opts.dynamical_decoupling.sequence_type = 'XX' # or 'XpXm', 'XY4'

# Manual DD via PassManager (for advanced control)
backend = service.backend('ibm_brisbane')
durations = InstructionDurations.from_backend(backend)

dd_sequence = [XGate(), XGate()] # XX sequence
dd_pass = PadDynamicalDecoupling(durations, dd_sequence)

pm = PassManager([dd_pass])
tqc_with_dd = pm.run(transpile(qc, backend))
```

Clifford Circuits & Stabiliser Simulation

```
from qiskit.quantum_info import Clifford
from qiskit import QuantumCircuit

# Clifford circuits: only H, S, CNOT, X, Y, Z, CZ
# Can be simulated EXPONENTIALLY faster with stabiliser formalism

qc = QuantumCircuit(4)
qc.h([0,1,2,3])
qc.cx(0,1); qc.cx(1,2); qc.cx(2,3)
qc.s(0); qc.z(2)

cliff = Clifford(qc) # exact stabiliser representation
print(cliff.tableau) # stabiliser tableau
print(cliff.to_dict())

# Reconstruct circuit from Clifford
qc_reconstructed = cliff.to_circuit()

# Check if two Cliffords are equivalent
cliff2 = Clifford(qc_reconstructed)
print(cliff == cliff2) # True
```

Unitary Synthesis

```
from qiskit.synthesis import OneQubitEulerDecomposer, TwoQubitBasisDecomposer
from qiskit.circuit.library import CXGate
from qiskit.quantum_info import random_unitary
import numpy as np

# Decompose arbitrary 1-qubit unitary into basis gates
decomposer_1q = OneQubitEulerDecomposer(basis='ZSX') # IBM native

# Random 1-qubit unitary
U = random_unitary(2).data
qc_1q = decomposer_1q(U) # circuit approximating U
print(qc_1q.draw('text'))

# Decompose arbitrary 2-qubit unitary into CX + 1-qubit gates
decomposer_2q = TwoQubitBasisDecomposer(CXGate())
U2 = random_unitary(4).data
qc_2q = decomposer_2q(U2) # at most 3 CX gates (KAK theorem)
print(f'CX count: {qc_2q.count_ops().get("cx", 0)}') # 0-3
```

Developer Exam Focus: Know that arbitrary 2-qubit unitaries require at most 3 CNOT gates (KAK decomposition theorem). This is a classic exam question about circuit complexity.

SECTION 4

Noise Models & Error Mitigation

Building Realistic Noise Models

```
from qiskit_aer.noise import (
NoiseModel, depolarizing_error, thermal_relaxation_error,
ReadoutError, pauli_error
)
from qiskit_ibm_runtime import QiskitRuntimeService

# ■■ From real backend properties ■■■■■■■■■■■■■■■■■■■■■■■■■■
service = QiskitRuntimeService(channel='ibm_quantum')
backend = service.backend('ibm_brisbane')

# Build noise model that mirrors real device
from qiskit_aer.noise import NoiseModel
noise_model = NoiseModel.from_backend(backend)
print(noise_model) # shows T1, T2, gate errors for each qubit

# ■■ Custom noise model from scratch ■■■■■■■■■■■■■■■■■■■■■■
nm = NoiseModel()

# Depolarising error: prob p of applying random Pauli
err_1q = depolarizing_error(0.001, 1) # 0.1% single-qubit error
err_2q = depolarizing_error(0.01, 2) # 1.0% two-qubit error
nm.add_all_qubit_quantum_error(err_1q, ['h','x','y','z','s','t','rx','ry','rz'])
nm.add_all_qubit_quantum_error(err_2q, ['cx', 'cz'])

# Thermal relaxation (T1/T2 decoherence)
t1, t2 = 80e-6, 120e-6 # T1=80us, T2=120us (typical IBM)
gate_time = 50e-9 # 50ns gate time
t_err = thermal_relaxation_error(t1, t2, gate_time)
nm.add_all_qubit_quantum_error(t_err, ['h', 'cx'])

# Readout error (measurement error)
# p(0|0): prob of measuring 0 when state is 0
# p(1|0): prob of measuring 1 when state is 0 (bit flip error)
ro_err = ReadoutError([[0.98, 0.02], [0.03, 0.97]])
nm.add_all_qubit_readout_error(ro_err)

# Run noisy simulation
from qiskit_aer import AerSimulator
noisy_sim = AerSimulator(noise_model=nm)
```

Zero-Noise Extrapolation (ZNE)

```
# ZNE: run circuit at scaled noise levels, extrapolate to zero noise
# Most practical error mitigation for NISQ devices

from mitiq import zne
```

```
from mitiq.interface.qiskit import from_qiskit
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
import numpy as np

# Setup noisy simulator
nm = NoiseModel()
nm.add_all_qubit_quantum_error(depolarizing_error(0.01, 1), ['h','x'])
nm.add_all_qubit_quantum_error(depolarizing_error(0.05, 2), ['cx'])
sim = AerSimulator(noise_model=nm)

# Circuit to mitigate
qc = QuantumCircuit(2)
qc.h(0); qc.cx(0,1)

# ZNE with Mitiq
def execute(circuit):
"""Execute circuit and return expectation of ZZ observable"""
from qiskit.quantum_info import SparsePauliOp, Statevector
tqc = transpile(circuit, sim)
# Use noisy statevector for expectation
from qiskit_aer import AerSimulator
sv_sim = AerSimulator(method='statevector', noise_model=nm)
tqc2 = circuit.copy()
tqc2.save_statevector()
job = sv_sim.run(transpile(tqc2, sv_sim))
sv = job.result().get_statevector()
from qiskit.quantum_info import Statevector
state = Statevector(sv)
op = SparsePauliOp('ZZ')
return state.expectation_value(op).real

# Manual ZNE: scale noise by 1x, 2x, 3x via gate folding
# Gate folding: replace G with G*G†*G (3x noise), etc.
noise_levels = [1, 2, 3]
expectation_vals = []
for scale in noise_levels:
qc_folded = qc.copy()
# Fold each gate (scale-1) extra times
# ... (Mitiq handles this automatically)
expectation_vals.append(execute(qc))

# Linear extrapolation to zero noise
coeffs = np.polyfit(noise_levels, expectation_vals, deg=1)
mitigated = np.polyval(coeffs, 0) # extrapolate to noise=0
print(f'Unmitigated: {expectation_vals[0]:.4f}')
```

```
print(f'Mitigated: {mitigated:.4f}')
print(f'Ideal: 1.0000 (for Bell state)')
```

Pauli Twirling

```
# Twirling: randomise coherent errors into incoherent (depolarising)
# Makes noise model more predictable and easier to mitigate

from qiskit_ibm_runtime import SamplerOptions

# Enable via Runtime options (simplest approach)
opts = SamplerOptions()
opts.twirling.enable_gates = True # twirl 2-qubit gates
opts.twirling.enable_measure = True # twirl measurements
opts.twirling.num_randomizations = 300 # number of random circuits
opts.twirling.shots_per_randomization = 'auto' # distribute shots

# Manual gate twirling (educational)
from qiskit.transpiler.passes import RandomizePauliOracle
# Each CX gate gets wrapped: (P1 x P2) @ CX @ (P1† x P2†)
# where P1, P2 are random Pauli operators
# Result: coherent error -> depolarising channel (easier to extrapolate)
```

ZNE + Twirling is the standard combination for NISQ error mitigation. Twirling first converts coherent errors to depolarising; ZNE then extrapolates away the remaining incoherent error.

SECTION 5

Variational Algorithms – VQE & QAOA Full Implementations

VQE — Variational Quantum Eigensolver (Production Grade)

```
from qiskit.circuit.library import TwoLocal, EfficientSU2
from qiskit.quantum_info import SparsePauliOp
from qiskit_ibm_runtime import EstimatorV2 as Estimator, Session
from qiskit_aer import AerSimulator
from qiskit import transpile
from scipy.optimize import minimize
import numpy as np

# ■■ Define Hamiltonian: H2 molecule (simplified) ■■■■■■■■■■
# Full H2 Hamiltonian (Jordan-Wigner mapping, STO-3G basis)
H2_hamiltonian = SparsePauliOp.from_list([
('II', -1.0523732),
('IZ', 0.3979374),
('ZI', -0.3979374),
('ZZ', -0.0112801),
('XX', -0.1809312),
])
# FCI ground state energy: -1.1373 Hartree

# ■■ Define ansatz ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# EfficientSU2: hardware-efficient ansatz, good for NISQ
ansatz = EfficientSU2(num_qubits=2, reps=2, entanglement='linear')
num_params = ansatz.num_parameters
print(f'Parameters: {num_params}') # 12

# ■■ Setup Estimator ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sim = AerSimulator()
estimator = Estimator(sim)

# ■■ Cost function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
call_count = [0]
energies = []

def cost_fn(params):
call_count[0] += 1
bound_circuit = ansatz.assign_parameters(params)
tqc = transpile(bound_circuit, sim)
job = estimator.run([(tqc, H2_hamiltonian)])
energy = job.result()[0].data.evs
energies.append(energy)
if call_count[0] % 20 == 0:
```

```
print(f'Iter {call_count[0]}: E = {energy:.6f} Ha')
return energy

# ■■ Optimise ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
x0 = np.random.uniform(-np.pi, np.pi, num_params)

# COBYLA: gradient-free, works well with noisy circuits
result_cobyla = minimize(cost_fn, x0, method='COBYLA',
options={'maxiter': 300, 'rhobeg': 0.5})

# SPSA: stochastic gradient, designed for noisy quantum systems
# (Simultaneous Perturbation Stochastic Approximation)
from qiskit_algorithms.optimizers import SPSA
spsa = SPSA(maxiter=300)

print(f'VQE minimum energy: {result_cobyla.fun:.6f} Ha')
print(f'FCI reference: -1.137274 Ha')
print(f'Error: {abs(result_cobyla.fun - (-1.137274))*1000:.2f} mHa')
```

QAOA — Full MaxCut Implementation

```
from qiskit.circuit.library import QAOAAnsatz
from qiskit.quantum_info import SparsePauliOp
from qiskit_ibm_runtime import EstimatorV2 as Estimator
from qiskit_aer import AerSimulator
from qiskit import transpile
from scipy.optimize import minimize
import numpy as np
import networkx as nx

# ■■ Problem: MaxCut on a 4-node graph ■■■■■■■■■■■■■■■■■■■■■
G = nx.Graph()
G.add_edges_from([(0,1,{'weight':1}), (1,2,{'weight':1}),
(2,3,{'weight':1}), (3,0,{'weight':1}),
(0,2,{'weight':1})])

# ■■ Build cost Hamiltonian: C = sum_(i,j) w_ij * (I - ZiZj)/2 ■■
pauli_list = []
n = G.number_of_nodes()
for u, v, data in G.edges(data=True):
w = data.get('weight', 1)
pauli_str = ['I'] * n
pauli_str[u] = 'Z'
pauli_str[v] = 'Z'
pauli_list.append((''.join(reversed(pauli_str)), -w/2))
pauli_list.append(('I'*n, w/2))

cost_op = SparsePauliOp.from_list(pauli_list)
print('Cost operator:', cost_op)

# ■■ QAOA Circuit (depth p=2) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
p = 2 # number of QAOA layers
qaoa = QAOAAnsatz(cost_operator=cost_op, reps=p)
print(f'QAOA parameters: {qaoa.num_parameters}') # 2*p = 4

# ■■ Optimise ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sim = AerSimulator()
estimator = Estimator(sim)

def qaoa_cost(params):
bound = qaoa.assign_parameters(params)
tqc = transpile(bound, sim)
job = estimator.run([(tqc, cost_op)])
return -job.result()[0].data.evs # negate: minimise -> maximise cut

x0 = np.array([0.5, 0.3, 0.8, 0.2]) # initial [gamma, beta, gamma, beta]
result = minimize(qaoa_cost, x0, method='COBYLA',
options={'maxiter': 200})

print(f'Optimal params: {result.x}')
print(f'Expected cut value: {-result.fun:.4f}')

# ■■ Sample optimal circuit to get cut assignment ■■■■■■■■■■
from qiskit_ibm_runtime import SamplerV2 as Sampler
sampler = Sampler(sim)
optimal_circuit = qaoa.assign_parameters(result.x)
optimal_circuit.measure_all()
tqc_m = transpile(optimal_circuit, sim)
job = sampler.run([tqc_m], shots=10000)
counts = job.result()[0].data.meas.get_counts()
best = max(counts, key=counts.get)
print(f'Best cut assignment: {best}') # e.g. '0101' or '1010'
```

Developer Exam: You must know QAOAAnsatz builds the alternating cost/mixer layers automatically. The optimal gamma/beta parameters initialisation matters — use p=1 analytical solution as warm start for p=2.

SECTION 6

Quantum Machine Learning – QNNs, Kernels, PennyLane

Quantum Neural Network with PennyLane + PyTorch

```
import pennylane as qml
from pennylane import numpy as np
import torch
from torch import nn
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler

# ■■ Device setup ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
n_qubits = 4
dev = qml.device('default.qubit', wires=n_qubits)

# ■■ QNN circuit ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@qml.qnode(dev, interface='torch')
def qnn_circuit(inputs, weights):
# Feature encoding: AngleEmbedding
qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
# Trainable layers: StronglyEntanglingLayers
qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
# Output: expectation of PauliZ on qubit 0
return qml.expval(qml.PauliZ(0))

# ■■ PyTorch integration ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
n_layers = 3
weight_shape = qml.StronglyEntanglingLayers.shape(n_layers=n_layers,
n_wires=n_qubits)

class HybridQNN(nn.Module):
def __init__(self):
super().__init__()
# Quantum layer
self.q_weights = nn.Parameter(torch.rand(weight_shape) * 2 * torch.pi)
# Classical pre/post processing
self.fc_in = nn.Linear(2, n_qubits) # 2 features -> 4 qubits
self.fc_out = nn.Linear(1, 2) # 1 QNN output -> 2 classes

def forward(self, x):
x = torch.tanh(self.fc_in(x)) * torch.pi # scale to [-pi, pi]
q_out = torch.stack([qnn_circuit(xi, self.q_weights) for xi in x])
q_out = q_out.unsqueeze(1) # [batch, 1]
return self.fc_out(q_out) # [batch, 2] logits
```

```
# ■■ Training ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
X, y = make_moons(n_samples=200, noise=0.1)
X = StandardScaler().fit_transform(X)
X_t = torch.FloatTensor(X)
y_t = torch.LongTensor(y)

model = HybridQNN()
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(50):
optimizer.zero_grad()
logits = model(X_t)
loss = loss_fn(logits, y_t)
loss.backward() # parameter shift rule auto-applied
optimizer.step()
if epoch % 10 == 0:
acc = (logits.argmax(1) == y_t).float().mean()
print(f'Epoch {epoch}: loss={loss:.4f}, acc={acc:.3f}')
```

Quantum Kernel SVM

```
from qiskit.circuit.library import ZZFeatureMap
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.algorithms import QSVC
from qiskit.primitives import StatevectorSampler
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

# ■■ Generate dataset ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
X, y = make_classification(n_samples=100, n_features=2,
n_informative=2, n_redundant=0,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# ■■ Quantum Feature Map ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ZZFeatureMap: encodes data in entangled Hilbert space
# reps=2: two layers of encoding circuits
feature_map = ZZFeatureMap(feature_dimension=2, reps=2)
print(feature_map.draw('text'))

# ■■ Quantum Kernel ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# K(x,x') = |<phi(x)|phi(x')>|^2
# phi(x): quantum state after applying feature_map to x
sampler = StatevectorSampler()
kernel = FidelityQuantumKernel(feature_map=feature_map,
sampler=sampler)
```

```
# Compute kernel matrix (expensive O(n^2) circuit evaluations)
K_train = kernel.evaluate(X_train) # [n_train x n_train]
K_test = kernel.evaluate(X_test, X_train) # [n_test x n_train]

# ■■ Classical SVM with quantum kernel ■■■■■■■■■■■■■■■■■■■■■■
from sklearn.svm import SVC
svm = SVC(kernel='precomputed')
svm.fit(K_train, y_train)
y_pred = svm.predict(K_test)
print(f'Quantum Kernel SVM accuracy: {accuracy_score(y_test, y_pred):.3f}')

# ■■ Or use QSVC (all-in-one) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
qsvc = QSVC(quantum_kernel=kernel)
qsvc.fit(X_train, y_train)
print(f'QSVC accuracy: {qsvc.score(X_test, y_test):.3f}')
```

Exam Focus: Know that FidelityQuantumKernel computes kernel entries as circuit FIDELITY (overlap squared). The quantum advantage requires that the feature space is classically hard to compute — ZZFeatureMap achieves this via entangling layers.

SECTION 7

Quantum Circuit Library – Built-in Algorithm Circuits

```
from qiskit.circuit.library import (
# Basis circuits
QFT, # Quantum Fourier Transform
PhaseEstimation, # Quantum Phase Estimation
GroverOperator, # Grover diffusion + oracle
# Feature maps for QML
ZZFeatureMap,
PauliFeatureMap,
# Ansatze for variational algorithms
TwoLocal,
EfficientSU2,
RealAmplitudes,
NLocal,
# Arithmetic
PhaseOracle,
IntegerComparator,
)

# ■■ QFT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
qft4 = QFT(num_qubits=4, approximation_degree=0, do_swaps=True)
print(qft4.num_qubits) # 4
print(qft4.depth()) # depth of circuit
iqft4 = QFT(4).inverse() # IQFT

# ■■ EfficientSU2 (most common VQE ansatz) ■■■■■■■■■■■■■■■■
ansatz = EfficientSU2(
num_qubits=4,
su2_gates=['ry', 'rz'], # rotation gates per qubit
entanglement='circular', # 'linear','full','circular','sca'
reps=3, # number of layers
skip_final_rotation_layer=False
)
print(f'Parameters: {ansatz.num_parameters}') # 4*(3+1)*2 = 32

# ■■ TwoLocal ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# More flexible than EfficientSU2
two_local = TwoLocal(
num_qubits=4,
rotation_blocks=['ry'], # single-qubit rotations
entanglement_blocks='cx', # entangling gates
entanglement='linear',
```

```
reps=2,
parameter_prefix='theta'
)

# ■■ RealAmplitudes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Special case: only real amplitudes, no imaginary parts
# Good for problems with real-valued ground states (e.g. some chemistry)
ra = RealAmplitudes(num_qubits=3, reps=2)

# ■■ PhaseEstimation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from qiskit.circuit.library import UnitaryGate
import numpy as np

# Estimate phase of eigenvalue of a unitary
# U|psi> = e^(2*pi*i*phi)|psi> -- estimate phi
U = UnitaryGate(np.array([[1,0],[0,np.exp(2j*np.pi*0.125)]]))
pe = PhaseEstimation(num_evaluation_qubits=3, unitary=U)
print(f'Phase estimation circuit depth: {pe.depth()}')
```

Developer Tip: Always use circuit library ansatze (EfficientSU2, TwoLocal) rather than building your own for exams and real projects. The exam will test your knowledge of their parameters: reps, entanglement, num_parameters.

SECTION 8

Advanced Transpilation – Custom PassManagers

```
from qiskit.transpiler import PassManager, CouplingMap, Layout
from qiskit.transpiler.passes import (
# Analysis passes
CheckMap, CheckGateDirection, Depth,
# Transformation passes
UnrollCustomDefinitions, BasisTranslator,
TrivialLayout, DenseLayout, SabreLayout,
BasicSwap, LookaheadSwap, SabreSwap,
Optimize1qGates, Optimize1qGatesDecomposition,
CXCancellation, CommutationAnalysis, CommutativeCancellation,
RemoveResetInZeroState, RemoveDiagonalGatesBeforeMeasure,
PadDynamicalDecoupling
)
from qiskit.transpiler.passes.synthesis import UnrollToBasis
from qiskit.circuit.equivalence_library import SessionEquivalenceLibrary
from qiskit import QuantumCircuit

# ■■ Custom PassManager for IBM backend ■■■■■■■■■■■■■■■■■■■■■■
# Replicates optimization_level=3 with manual control

from qiskit_ibm_runtime import QiskitRuntimeService
service = QiskitRuntimeService(channel='ibm_quantum')
backend = service.backend('ibm_brisbane')

coupling_map = CouplingMap(backend.coupling_map)
basis_gates = backend.basis_gates # ['cx','id','rz','sx','x','measure']

qc = QuantumCircuit(4)
qc.h([0,1,2,3]); qc.cx(0,2); qc.cx(1,3); qc.cx(2,3); qc.measure_all()

# Stage 1: Layout (qubit mapping)
layout_pm = PassManager([
SabreLayout(coupling_map, max_iterations=4), # best layout heuristic
])

# Stage 2: Routing (SWAP insertion)
routing_pm = PassManager([
SabreSwap(coupling_map, heuristic='decay'), # best routing heuristic
])

# Stage 3: Translation to basis gates
translation_pm = PassManager([
UnrollCustomDefinitions(SessionEquivalenceLibrary, basis_gates),
BasisTranslator(SessionEquivalenceLibrary, basis_gates),
])
```

```
# Stage 4: Optimisation
optimisation_pm = PassManager([
Optimize1qGatesDecomposition(basis=basis_gates),
CXCancellation(),
CommutationAnalysis(),
CommutativeCancellation(),
RemoveResetInZeroState(),
RemoveDiagonalGatesBeforeMeasure(),
])

# Run all stages
tqc = layout_pm.run(qc)
tqc = routing_pm.run(tqc)
tqc = translation_pm.run(tqc)
tqc = optimisation_pm.run(tqc)

print(f'Original depth: {qc.depth()}')
print(f'Transpiled depth: {tqc.depth()}')
print(f'SWAP gates added: {tqc.count_ops().get("swap", 0)}')

# ■■ StagedPassManager (Qiskit v1 recommended approach) ■■
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
pm = generate_preset_pass_manager(
optimization_level=3,
backend=backend,
initial_layout=[0, 1, 2, 3] # force specific qubit mapping
)
tqc2 = pm.run(qc)
```

SabreLayout + SabreSwap is the current state-of-the-art for layout and routing on IBM hardware. For small circuits (<10 qubits), TrivialLayout + BasicSwap may be faster to compile and nearly as good.

SECTION 9

Quantum Information – Fidelity, Entanglement, Tomography

```
from qiskit.quantum_info import (
Statevector, DensityMatrix, Operator,
state_fidelity, average_gate_fidelity,
entropy, entanglement_of_formation,
partial_trace, Pauli, SparsePauliOp,
random_statevector, random_unitary
)
from qiskit import QuantumCircuit
import numpy as np

# ■■ State fidelity ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
qc_ideal = QuantumCircuit(2)
qc_ideal.h(0); qc_ideal.cx(0,1)
sv_ideal = Statevector(qc_ideal)

# Slightly noisy state (manually perturbed)
sv_noisy = Statevector([0.71, 0, 0, 0.69+0.05j])
sv_noisy = sv_noisy / sv_noisy.norm()

fidelity = state_fidelity(sv_ideal, sv_noisy)
print(f'State fidelity: {fidelity:.4f}') # 1.0 = perfect

# ■■ Density matrix and purity ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
dm = DensityMatrix(qc_ideal)
print(f'Purity: {dm.purity():.4f}') # 1.0 = pure state
print(f'Trace: {dm.trace():.4f}') # always 1.0

# Mixed state example
dm_mixed = DensityMatrix.from_label('mixed') # maximally mixed
print(f'Mixed purity: {dm_mixed.purity():.4f}') # 0.5 for 2-qubit

# ■■ Von Neumann entropy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Measures entanglement: 0 = product state, 1 = maximally entangled
bell_dm = DensityMatrix(qc_ideal)
# Trace out qubit 1 to get reduced density matrix of qubit 0
rho_0 = partial_trace(bell_dm, [1]) # trace out qubit 1
S_vn = entropy(rho_0, base=2) # Von Neumann entropy
print(f'Entanglement entropy: {S_vn:.4f}') # 1.0 for Bell state

# ■■ Operator fidelity ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from qiskit.circuit.library import HGate
ideal_H = Operator(HGate())
# Simulate slightly noisy H gate
noisy_H_mat = ideal_H.data + 0.01 * np.random.randn(2,2)
```

```
noisy_H_mat = noisy_H_mat / np.linalg.norm(noisy_H_mat) * np.sqrt(2)
noisy_H = Operator(noisy_H_mat)
gate_fid = average_gate_fidelity(noisy_H, ideal_H)
print(f'Gate fidelity: {gate_fid:.4f}')

# ■■ State Tomography ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from qiskit_experiments.library import StateTomography

qc_prep = QuantumCircuit(2)
qc_prep.h(0); qc_prep.cx(0, 1) # Bell state

# StateTomography measures in all Pauli bases to reconstruct rho
exp = StateTomography(qc_prep)
# exp.run(backend).block_for_results() on real hardware
# For simulator:
from qiskit_aer import AerSimulator
sim_data = exp.run(AerSimulator(), shots=4096).block_for_results()
rho = sim_data.analysis_results('state').value
print(f'Reconstructed state fidelity: {state_fidelity(rho, bell_dm):.4f}')
```

Exam: The Developer exam tests whether you can SELECT the right quantum information tool. state_fidelity() for states, average_gate_fidelity() for gates, entropy() for entanglement, StateTomography for full state reconstruction.

SECTION 10

Hybrid Architecture Patterns – Architect-Level Design

As a Principal Architect, the Developer exam asks you to design and evaluate complete hybrid quantum-classical systems — not just write code snippets.

The Variational Hybrid Loop (Canonical Pattern)

```
# This is the TEMPLATE for all variational quantum algorithms
# VQE, QAOA, QNN training all follow this pattern

import numpy as np
from qiskit_ibm_runtime import EstimatorV2 as Estimator, Session
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.circuit.library import EfficientSU2
from qiskit.quantum_info import SparsePauliOp
from qiskit import transpile
from scipy.optimize import minimize

class VariationalHybridLoop:
"""
Generic hybrid quantum-classical variational loop.
Classical optimizer drives quantum circuit parameter updates.
"""
def __init__(self, hamiltonian, ansatz, backend):
self.hamiltonian = hamiltonian
self.ansatz = ansatz
self.backend = backend
self.history = []

def run(self, x0, method='COBYLA', maxiter=200):
with Session(backend=self.backend) as session:
estimator = Estimator(session=session)

def cost(params):
bound = self.ansatz.assign_parameters(params)
tqc = transpile(bound, self.backend)
job = estimator.run([(tqc, self.hamiltonian)])
energy = job.result()[0].data.evs
self.history.append(energy)
return energy

result = minimize(cost, x0, method=method,
options={'maxiter': maxiter})
return result

# Usage
```

```
H = SparsePauliOp.from_list([('ZZ', -1), ('XX', 0.5), ('YY', 0.5)])
ansatz = EfficientSU2(2, reps=2)
service = QiskitRuntimeService(channel='ibm_quantum')
backend = service.least_busy(min_num_qubits=2)

loop = VariationalHybridLoop(H, ansatz, backend)
x0 = np.random.uniform(0, 2*np.pi, ansatz.num_parameters)
result = loop.run(x0)
print(f'Ground state energy: {result.fun:.6f}')
```

Quantum-Classical Pipeline for Production

Problem Encoding: Map domain problem to quantum Hamiltonian or oracle. This step is often the hardest — a poor encoding can negate any quantum advantage. Use Jordan-Wigner for fermionic systems; QUBO for combinatorial problems.

Hardware vs Simulator: Use simulators for development and debugging (free, fast). Switch to real hardware for final benchmarking. Use noise models to bridge the gap. Always check qubit connectivity before layout decisions.

Error Mitigation Strategy: For NISQ: ZNE + Twirling is the standard combo. For critical accuracy: Probabilistic Error Cancellation (PEC). Always include measurement error mitigation (readout calibration).

Shot Budget: Estimator circuits: 1000-10000 shots per expectation value. Sampler circuits: 4096-8192 shots for distribution accuracy. With ZNE at 3 noise levels: multiply shot budget by 3.

Classical Optimiser Selection: Low noise, few params: BFGS (gradient-based). High noise, many params: SPSA (stochastic, noise-robust). Landscape exploration needed: CMA-ES or Bayesian optimisation.

Architecture Anti-Pattern: Never put quantum execution in a real-time inference loop. Quantum circuit overhead (queuing, execution, readout) is measured in seconds to minutes. Design for batch or async execution.

SECTION 11

Practice Exam – 50 Hard Questions with Explanations

Developer-level questions require deeper reasoning than Associate questions. Many require you to trace through code mentally and predict output, or choose the correct architectural decision.

Q1. Which Qiskit Runtime v2 primitive is used for VQE?

A) SamplerV2

B) EstimatorV2

C) ExecutorV2

D) MeasurerV2

Answer: B — VQE requires expectation values of Hamiltonians. EstimatorV2 computes expectation values. SamplerV2 returns bit-string distributions.

Q2. In EstimatorV2, job.result()[0].data.evs returns:

A) A dictionary of counts

B) A BitArray object

C) The expectation value (scalar or array)

D) A Statevector

Answer: C — evs = expectation values. The result structure changed in Runtime v2 — evs is under .data, not .values.

Q3. EfficientSU2(n_qubits=3, reps=2).num_parameters equals:

A) 6

B) 12

C) 18

D) 24

*Answer: C — EfficientSU2: (reps+1) * n_qubits * 2 rotation params = 3*3*2 = 18.*

Q4. Which error mitigation technique converts coherent errors to incoherent?

A) ZNE

B) PEC

C) Pauli Twirling

D) Readout calibration

Answer: C — Twirling randomises phases of coherent errors, converting them to depolarising (incoherent) errors.

Q5. ZNE extrapolates expectation values by:

A) Running the circuit at zero noise directly

B) Running at multiple noise levels and fitting to zero

C) Subtracting the noise floor from results

D) Averaging over many random circuits

Answer: B — ZNE: run at 1x, 2x, 3x noise (via gate folding), fit a polynomial, extrapolate to noise=0.

Q6. What is the maximum number of CNOT gates to decompose any 2-qubit unitary?

A) 1

B) 2

C) 3

D) 4

Answer: C — KAK theorem: any 2-qubit unitary decomposes into at most 3 CNOT gates plus single-qubit rotations.

Q7. partial_trace(dm, [1]) on a 2-qubit density matrix:

A) Traces out qubit 0

B) Traces out qubit 1

C) Returns trace of qubit 1 (scalar)

D) Entangles qubits 0 and 1

Answer: B — partial_trace(state, qargs) traces out the qubits listed in qargs. [1] traces out qubit 1.

Q8. entropy(rho_0, base=2) returns 1.0 for the reduced state of a Bell pair because:

A) The Bell pair is a mixed state

B) Qubit 0 is maximally mixed after tracing out qubit 1

C) The circuit has 2 qubits

D) Von Neumann entropy is always 1.0

Answer: B — Bell state reduced density matrix is I/2 (maximally mixed). VN entropy of maximally mixed 1-qubit state = 1 ebit.

Q9. Session mode is recommended for VQE because:

A) It provides exclusive QPU access

B) It reduces compilation overhead

C) It keeps backend connection open, reducing queue overhead per iteration

D) It enables error correction

Answer: C — Sessions share QPU context across jobs, dramatically reducing latency for iterative algorithms.

Q10. QAOAAnsatz(cost_operator=C, reps=p).num_parameters equals:

A) p

B) 2p

C) p*n_qubits

D) 2*p*n_qubits

Answer: B — QAOA has p gamma parameters (cost layer) + p beta parameters (mixer layer) = 2p total.

Q11. NoiseModel.from_backend(backend) builds a noise model based on:

A) The backend's coupling map only

B) T1, T2, gate error rates, and readout errors from backend properties

C) Only readout errors

D) A fixed depolarising error of 1%

Answer: B — from_backend() reads all calibration data: T1/T2 times, gate error rates per qubit, readout errors.

Q12. state_fidelity(sv1, sv2) returns 1.0 when:

A) sv1 and sv2 have the same number of qubits

B) sv1 and sv2 represent identical quantum states

C) Both states are pure

D) Both states have zero entropy

Answer: B — Fidelity = ||^2 = 1 iff states are identical (up to global phase).

Q13. Which FidelityQuantumKernel parameter controls the feature Hilbert space?

A) sampler

B) feature_map

C) num_qubits

D) reps

Answer: B — The feature_map circuit defines how classical data is embedded in Hilbert space. Different maps = different kernels.

Q14. Dynamical Decoupling inserts gates during:

A) Gate operations

B) Measurement

C) Idle periods between gate operations

D) Circuit compilation

Answer: C — DD fills idle time slots with symmetric gate sequences (XX, XY4) to echo out noise during qubit idling.

Q15. generate_preset_pass_manager(optimization_level=3, backend=b) is preferred over transpile() because:

A) It runs faster

B) It gives more control via stageable pipeline

C) It auto-selects best layout and routing passes for the backend

D) Both B and C

Answer: D — generate_preset_pass_manager() exposes stage-wise PM (layout, routing, translation, optimisation) AND auto-configures for backend.

Official Practice: IBM Quantum Learning at learning.quantum.ibm.com has graded assessments mapped to C1000-171 domains. Complete ALL learning paths before exam day, especially 'Quantum Machine Learning' and 'Advanced Circuits' modules.

SECTION 12

Master Reference & Resources

Class/Module	Method/Attribute	Notes
EstimatorV2	.run(pubs)	pubs = list of (circuit, observable) tuples
EstimatorV2	result[i].data.evs	Expectation value for i-th PUB
SamplerV2	.run([tqc], shots=N)	List of transpiled circuits
SamplerV2	result[i].data.meas.get_counts()	Counts for classical register 'meas'
Session	with Session(backend) as s:	Context manager; closes on exit
NoiseModel	.from_backend(b)	Build from real device calibration
FidelityQuantumKernel	.evaluate(X,Y)	Compute kernel matrix K[i,j]
QAOAAnsatz	reps=p	2p parameters total (gamma+beta)
EfficientSU2	num_parameters	(reps+1) * n_qubits * 2
state_fidelity	(sv1, sv2)	Returns ||^2
partial_trace	(dm, qargs)	Trace out qubits in qargs list
entropy	(rho, base=2)	Von Neumann entropy in bits
Statevector	.expectation_value(op)	Direct exp value; no shots
Operator	.is_unitary()	Verify gate is valid quantum operation
generate_preset_pass_manager	optimization_level, backend	Recommended transpilation API (v1.x)

Key Resources for C1000-171:

- IBM Quantum Learning: learning.quantum.ibm.com (complete ALL paths)
- Qiskit documentation: docs.quantum.ibm.com
- qiskit-ibm-runtime docs: qiskit.github.io/qiskit-ibm-runtime
- PennyLane QML tutorials: pennylane.ai/qml (for QML section)
- IBM C1000-171 exam page: ibm.com/training/certification/C1000171

You are ready to sit the Developer exam. You have mastered Runtime v2, variational algorithms, error mitigation, QML, and hybrid architecture. Run every code block. Build at least one VQE and one QAOA from scratch. Then book the exam — you're prepared.